

William Douglass

Dr. Ruoti

COSC 583

Password Cracking Report

I started the Password Cracking project in the late evening of Monday, November 3rd. I managed to crack all 20 passwords in about 26 hours. Note that this was not 26 hours of consistent running time but including a lot of configuration to ensure correct usage. I chose hashcat as my preferred password cracking tool because I found it to be easier to install than JohnTheRipper. When I clicked on the JohnTheRipper link, it directed me to the GitHub, which I did not feel like reading and messing with. Therefore, I chose hashcat, which had a link that easily downloaded it. Figuring out how to use hashcat was not the easiest, but I found a video online (link at the end of the report) that explained and demonstrated the capabilities of hashcat quite well. That mixed with the existing documentation made learning how to use hashcat simple.

Hardware wise, I initially started using my laptop, which sports 8GB of RAM and a clock speed of 1.80 GHz. My laptop does not have a dedicated GPU and was slower than what I ended up using, which was the Hydra/Tesla lab machines. Still, my laptop was cracking at a rate of around 130,000 hashes per second once hashcat was able to multithread the operation. This only happened when the search space was large enough, which only occurred starting at lowercase passwords at a length of five or longer. I ultimately switched over to the Hydra/Tesla lab machines upon remembering that they had 32GB of RAM and presumably, dedicated GPUs that could crack at a rate one or two orders of magnitude higher than my laptop. Once fully multi-threaded, the Hydra/Tesla machines were cracking at a rate of around 7,000,000 hashes per second, with a low of around 6,250,000 hashes per second and a high of around 8,000,000 hashes per second.

Passwords and guessing strategies (passwords are split by type of password).

Lowercase: ta, lsv, uddg, lkyru, and ugrknq

The first 4 were cracked using the -i option and “?!l?!l?!l?!l?!l” with the ‘-a’ option set to 3. The last lowercase password was found using only “?!l?!l?!l?!l”. This was running on a separate lab machine while also attempting to crack the alphanumeric passwords.

Alphanumeric: SJ, lr4, 31w4, 2c8zc, and 7OgTHT

The first 3 were cracked using the -i option and “?a?a?a?a?a?a?a”. This was before I realized “a” also searched through non-alphanumeric characters. That was changed to “?1?1?1?1?1?1?1”, where the -1 option was defined as ?l?u?d (the alphanumeric characters). These were the most time consuming, and I ended up running “?1?1?1?1?1” and “?1?1?1?1?1?1” on two separate lab machines.

Dictionary: cincinnati, revolutionary, longitude, forwarding, and increased

Hashcat has an option to perform a dictionary attack by setting the ‘-a’ option to 0. I provided the “wordlist.txt” provided by Dr. Ruoti. These passwords were cracking almost immediately.

Dictionary + l33t: c0mp4ni35, 41t3rn4t3, 31iz4b3th, int3rn4ti0n411y, and 14b0r4t0ri35

Hashcat has a ‘-r’ option that can be used to apply certain permutations to passwords. I supplied it a file titled “leetpower.rule”, which was created using a C++ script I created called “powerset.cpp”, which accepts a base file I titled “leet.rule”, which lists twenty substitution rules that turned certain letters into numbers. After some testing, I realized that I needed the powerset of the initial rules that I created, hence the C++ which generated a massive file with $2^{20} - 1$ rules. The search space was only $2241 * (2^{20} - 1)$, which is around 2.3 billion and could be iterated through in around five minutes on the lab machines.

Link to hashcat tutorial:

<https://www.youtube.com/watch?v=EfqJCKWtGiU>

Questions

1. The Hydra/Tesla machines have an average cracking speed of 7,000,000 hashes/s.
The possible number of alphanumeric passwords of lengths 6,8,10, and 12 are 62^6 , 62^8 , 62^{10} and 62^{12} respectively.

Math:

$$62^6 / 7.000.000 = 8114 \text{ seconds} / 3600 = 2.25 \text{ hours}$$

$$62^8 / 7.000.000 = 31191444 \text{ seconds} / 3600 = 8664 \text{ hours or } 361 \text{ days}$$

$$62^{10} / 7.000.000 = 119899909410 \text{ seconds} / 3600 = 33305530 \text{ hours}$$

$$\text{or } 1387730 \text{ days}$$

$$\text{or } 3802 \text{ years}$$

$$62^{12} / 7.000.000 = 4.6089525e+14 \text{ seconds} / 3600 = 128026458825 \text{ hours}$$

$$\text{or } 5334435784 \text{ days}$$

$$\text{or } 14,614,892 \text{ years}$$

Homo habilis, an ancestor to today's humans, evolved ~2,000,000 years ago.

That is how long it would take to iterate through the entire search space for each password class. Divide everything by 2 to get the expected amount of time it would take to get the password (i.e., 50% chance of getting the password).

2. Assuming cracking speed of 2.5 billion checks/second

Math:

$$62^6 / 2.500.000.000 = 22.7 \text{ seconds}$$

$$62^8 / 2.500.000.000 = 87336 \text{ seconds} / 3600 = 24.3 \text{ hours}$$

$$62^{10} / 2.500.000.000 = 335719746 \text{ seconds} / 3600 = 93255 \text{ hours}$$

$$\text{or } 3885 \text{ days}$$

$$\text{or } 10.7 \text{ years}$$

$$62^{12} / 2.500.000.000 = 1.2905067e+12 \text{ seconds} / 3600 = 358474085 \text{ hours}$$

$$\text{or } 14936420 \text{ days}$$

$$\text{or } 40921.7 \text{ years}$$

3. Based on my experience in this project and other things I've done related to passwords, I'm torn with the usefulness and accuracy of the password security meter. I don't think the password security is the most accurate, especially against dictionary attacks and dictionary attacks that make small permutations that most people do to their passwords in the mistaken belief that it improves the password's security. Because people are not random, these small changes are predictable, and rulesets can be easily created to search through these simple permutations. However, I think for the most vulnerable, the password security meter is the most useful, since older people who may have less secure passwords could be convinced to swap out of a very insecure for one that might be less insecure. From the results of my experiment and some additional calculations above, I would recommend a minimum password length of 10. Most dedicated attackers could get their hands on a high-end GPU that guess at 2.5 billion checks per second, so forcing them to wait 10 years before stealing a password sounds great. It also provides some security in the future, since GPUs will presumably get better, unless we hit some physical limitation, so a GPU released 10 years from now would presumably still have to work for a year before getting the password. The only issue is memorability, because a completely random alphanumeric password that is 10 characters long may be difficult to memorize compared to a slightly long lowercase password.
4. It only improves password security in that it slows down the hashing process. MD5 is faster than SHA512 because it is a simpler algorithm. More time to hash makes checks long to perform, reducing the checks per second.
5. Salting improves password security by preventing rainbow table attacks. Instead of instantly looking up the hash of an unsalted password, an attacker has to brute-force the password instead, even if the salt is known.
6. Yes. My thinking is that only way to get password hashes is to steal the password database or file and would presumably be fairly difficult for an attacker to steal the password file online. It's still important to think about, especially for passwords intended to secure sensitive information, but for a website like "recipesaver.com" that does not store any sensitive information, offline password security may not be that important to consider, especially if you do not reuse your passwords.